

This is actually one of the most important questions that judges or interviewers ask:

"Where did AI help in your project?"

A common mistake is saying, *"I used ChatGPT to generate the code."* Instead, explain **how AI improved the development process** while you still understood, reviewed, and integrated the result.

Previous Project vs AI-Assisted Project

Feature	Previous Project	AI-Assisted Project
API Type	Unary RPC	Server Streaming RPC
Communication	One request → One response	One request → Multiple responses
Protobuf	Written manually	AI generates the initial contract from requirements
Server Code	Written manually	AI scaffolds the server structure
Client Code	Written manually	AI scaffolds the client structure
Documentation	Manual	AI drafts documentation, refined by developer
Architecture	Simple	Modular and easier to extend
Development Speed	Slower	Faster initial development
Maintenance	More manual work	Easier because of consistent scaffolding

Traditional Development

Imagine a teacher says:

"Create a Greeting Service."

Developer's Work

Think about requirements



↓

Design protobuf

↓

Write proto

↓

Generate code

↓

Write server

↓

Write client

↓

Test

↓

Fix errors

↓

Write documentation

Everything starts from scratch.

AI-Assisted Development

Now the developer uses an AI assistant.

Requirement



↓

AI generates .proto

↓

Developer reviews it

↓

AI scaffolds server

↓

Developer customizes logic

↓

AI scaffolds client

↓

Developer tests

↓

AI helps document the project

The AI accelerates the repetitive work, but the developer remains responsible for the design and correctness.

Visual Comparison

Previous Project

Developer

↓

Write Everything

↓

Test

↓

Deploy



AI-Assisted Project

Developer



↓

AI Assistant

↓

.proto Generated

↓

Server Generated

↓

Client Generated

↓

Developer Reviews

↓

Testing

↓

Deployment

Difference in the Protobuf File

Previous Project

You manually wrote:

⌄ protobuf



```
service GreetingService {  
  rpc SayHello(HelloRequest) returns (HelloReply);  
}
```

You decided:

- Service name
- Request
- Response

- Messages

AI-Assisted Project

You give a prompt such as:

"Create a notification service using server streaming."

AI proposes:

`</> protobuf`



```
service NotificationService {  
  rpc Subscribe(NotificationRequest)  
  returns (stream Notification);  
}
```

You then review and adjust it if needed.

Difference in Server Code

Previous

Write function



↓

Register service

↓

Handle request

↓

Return response

AI-Assisted

Prompt:

"Generate a TypeScript gRPC server using this protobuf."

AI produces the boilerplate:

⌞ TypeScript

```
server.addService(...)  
  
function subscribe(...) {  
}
```



You add the business logic.

Difference in Client

Previous

Create client

↓

Connect

↓

Call RPC

↓

Receive one response



AI-Assisted

Prompt:

"Generate a TypeScript client for a server streaming API."

AI creates:

⌞ TypeScript

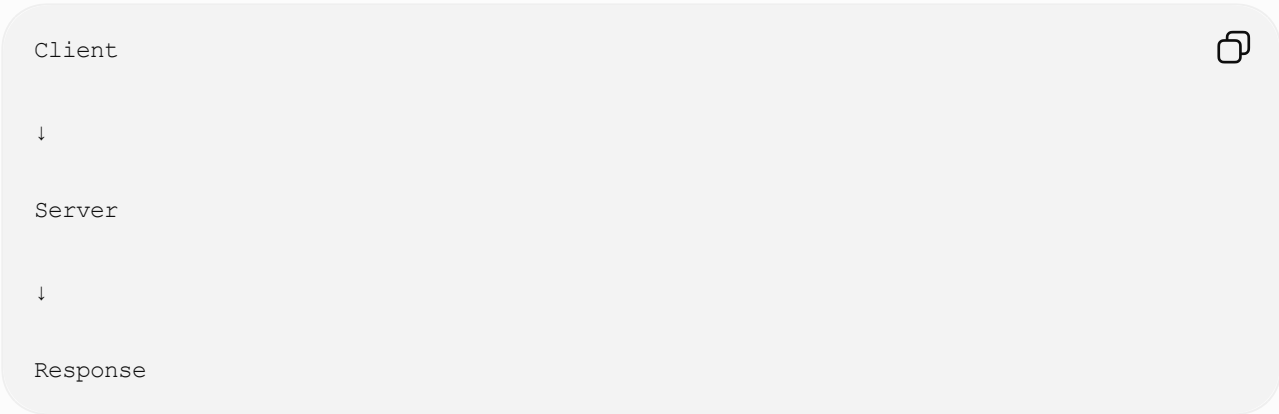
```
stream.on("data")  
  
stream.on("end")
```



You verify it works and adapt it to your application.

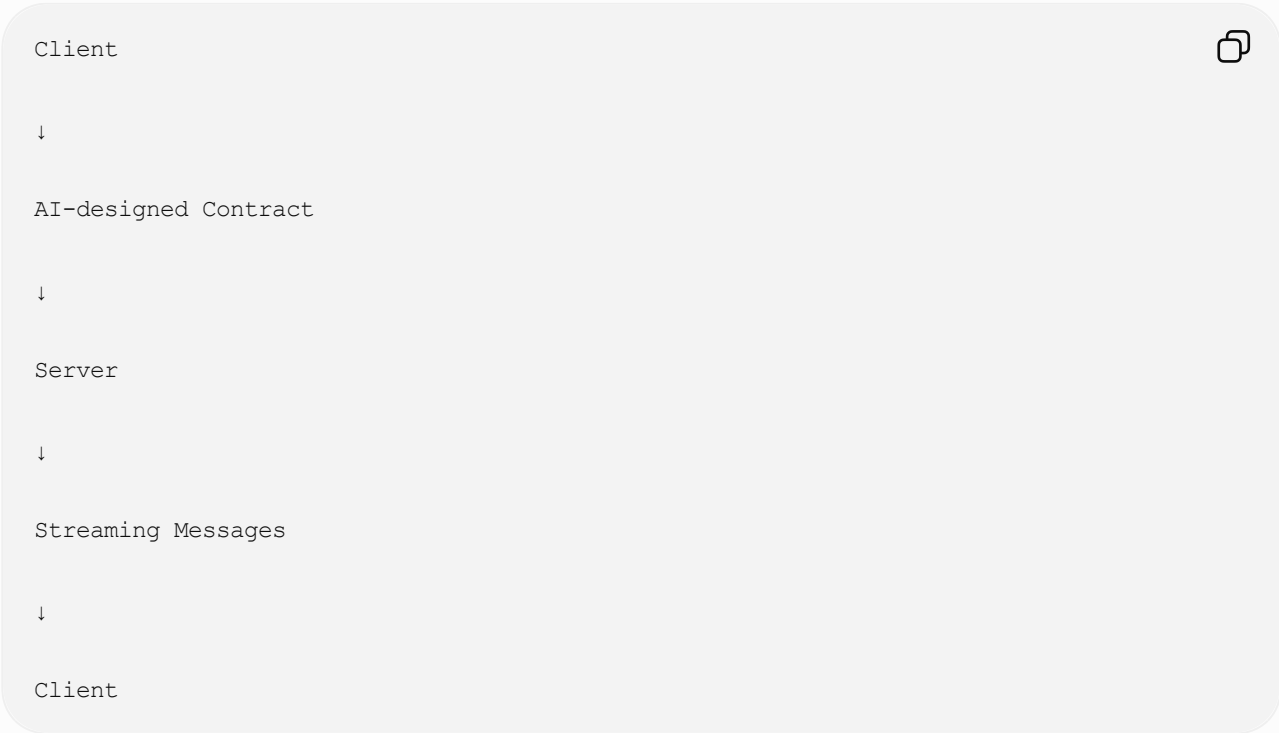
Difference in Architecture

Previous



One interaction.

AI-Assisted



Multiple updates over a single connection.

Difference in Development Time

Task	Previous	AI-Assisted
Design protobuf	30 min	5 min (draft)

Task	Previous	AI-Assisted
Server scaffold	45 min	10 min (draft)
Client scaffold	30 min	10 min (draft)
Documentation	45 min	10 min (draft)

These are illustrative estimates—actual times vary depending on experience and project complexity.

AI Prompts Used

Prompt 1

Generate a Protocol Buffer contract for a notification service using server streami



Prompt 2

Generate a TypeScript gRPC server using @grpc/grpc-js.



Prompt 3

Generate a TypeScript client that consumes a server stream.



Prompt 4

Refactor the project into a modular folder structure.



Prompt 5

Generate documentation explaining how the streaming service works.



Benefits of AI Assistance

- Speeds up repetitive tasks such as scaffolding.
- Produces a consistent initial project structure.
- Helps generate documentation and comments.
- Makes it easier to explore alternative API designs.
- Lets developers spend more time on business logic and testing.

Remember: AI-generated code should always be reviewed and tested before use.

A Good 2-Minute Explanation for Judges

"In the first version, I manually designed the Protocol Buffer file and wrote the client and server code myself. It demonstrated basic unary communication where one request returns one response.

In the improved version, I used AI to help generate the initial protobuf contract and scaffold the TypeScript server and client. I reviewed and modified the generated code to fit the project requirements. I also changed the API to use **server streaming**, so a single client request can receive multiple updates over time. AI reduced the time spent writing repetitive boilerplate, while I focused on understanding the architecture, implementing the business logic, testing the application, and validating that it behaved correctly."

This explanation clearly distinguishes **AI as a development assistant** rather than a replacement for engineering judgment, which is typically what judges and interviewers are looking for.